COLLEGE PROJECT DOCUMENTATION

Library Management System

Complete Backend Source Code & Architectural Reference

This document compiles all backend source codes for the **Advanced Programming Practice** laboratory evaluation. It includes full domain entities demonstrating Object-Oriented Programming (OOP) concepts, Spring Data JPA repositories, transactional business services for Step 10 issue & return logic (Rs. 5/day fine rule), Step 11 dynamic dashboard statistics APIs, and a Viva oral exam cheat sheet.

★ Evaluation Highlights & Step Compliance

Step 10: Implemented Book Management (Add, Edit, Delete, Search, Availability), Member Management (Add, Edit, Delete, Search), Issue workflow (14-day due date), and Return workflow (Late days × ₹5 fine).
Step 11: Dynamic Dashboard APIs delivering live metrics: **Total Books: 100 | Available: 75 | Issued: 25 | Members: 50**.
Step 12: Single-command Spring Boot execution serving integrated Web UI on `http://localhost:8080`.

SUBJECT / COURSE
Advanced Programming Practice

FRAMEWORK & LANGUAGE
Java 17 (LTS) • Spring Boot 3.2.5

DATABASE & ORM
Spring Data JPA • H2 Database

APPLICATION SERVER
Apache Tomcat (Port 8080)

Table of Contents & OOP Concepts Mapping

File Name	Layer / Category	OOP Concept / Technical Focus	Role in System
BaseEntity.java	Domain Entities	OOP Inheritance (@MappedSuperclass)	Shared ID and automated LocalDateTime audit timestamps for all entities.
Person.java	Domain Entities	OOP Inheritance & Encapsulation	Abstract parent entity providing common identity (name, email, phone) to Member.
Book.java	Domain Entities	Domain Modeling & Encapsulation	Catalog representation with private fields, validation, and markIssued() / markReturned() state logic.
Member.java	Domain Entities	Multilevel Inheritance	Extends Person (which extends BaseEntity) to represent students and faculty members.
Transaction.java	Domain Entities	Entity Relationships (@ManyToOne)	Links Book and Member entities to record loan issue, due date, return date, and late fines.
FineCalculator.java	Fine Calculation & Strategy	OOP Abstraction & Interface	Interface defining the contract for late days calculation and monetary fine calculation.
StandardFineCalculator.java	Fine Calculation & Strategy	OOP Polymorphism	Concrete implementation calculating late days * Rs. 5 as mandated by Step 10 return workflow.
BookRepository.java	Persistence Layer (Spring Data JPA)	JPA Repository & JPQL	Custom JPQL query for full-text searching across Title, Author, Category, and ISBN.
MemberRepository.java	Persistence Layer (Spring Data JPA)	Data Query Abstraction	Member search and uniqueness checking methods.
TransactionRepository.java	Persistence Layer (Spring Data JPA)	JPA Aggregations	Queries for active loans, overdue loans, and dynamic sum of all collected fines.

File Name	Layer / Category	OOP Concept / Technical Focus	Role in System
BookService.java	Service Layer (Step 10 & 11 Logic)	Interface Decoupling	Contract for Step 10: Add, Edit, Delete, Search, and Check Availability operations.
BookServiceImpl.java	Service Layer (Step 10 & 11 Logic)	Business Logic & Collections	Handles book CRUD, safe deletion checks, and catalog availability management.
MemberService.java	Service Layer (Step 10 & 11 Logic)	Interface Decoupling	Contract for Step 10: Add, Edit, Delete, and Search member records.
MemberServiceImpl.java	Service Layer (Step 10 & 11 Logic)	Business Validation	Ensures member ID uniqueness and prevents deleting members with outstanding loans.
TransactionService.java	Service Layer (Step 10 & 11 Logic)	Transaction Contract	Defines the Step 10 Issue Book and Return Book workflows.
TransactionServiceImpl.java	Service Layer (Step 10 & 11 Logic)	ACID Transactions & Workflow	Executes Issue logic (available=false, 14-day due date) and Return logic (available=true, Rs. 5/day fine).
DashboardService.java	Service Layer (Step 10 & 11 Logic)	Metrics Contract	Contract for Step 11 dynamic dashboard metrics.
DashboardServiceImpl.java	Service Layer (Step 10 & 11 Logic)	Java Streams & Aggregation	Dynamically computes Total Books (100), Available (75), Issued (25), Members (50) from live database.
BookController.java	REST Controllers (APIs)	RESTful API Design	Exposes /api/books endpoints for CRUD, search, and availability check.
MemberController.java	REST Controllers (APIs)	RESTful API Design	Exposes /api/members endpoints for member registration, search, and management.
TransactionController.java	REST Controllers (APIs)	RESTful API Design	Exposes /api/transactions endpoints for /issue, /return, and active loans.
DashboardController.java	REST Controllers (APIs)	Dynamic API Data	Exposes /api/dashboard/stats returning live JSON metrics for Step 11 dashboard.

File Name	Layer / Category	OOP Concept / Technical Focus	Role in System
GlobalExceptionHandler.java	Exception Handling & DTOs	**Centralized Error Handling**	@RestControllerAdvice intercepting custom exceptions to return consistent JSON errors.
IssueRequest.java	Exception Handling & DTOs	**Data Transfer Object (DTO)**	Encapsulates request payload for issuing books.
ReturnResponse.java	Exception Handling & DTOs	**Data Transfer Object (DTO)**	Encapsulates return details, late days calculation, and fine breakdown.
DashboardStats.java	Exception Handling & DTOs	**Data Transfer Object (DTO)**	Holds the Step 11 metrics payload.
DataInitializer.java	Application Startup & Configuration	**CommandLineRunner Seed Data**	Automatically populates 100 books, 50 members, and 25 loans for the live presentation demo.
LibraryManagementApplication.java	Application Startup & Configuration	**Main Entry Point**	@SpringBootApplication bootstrapping Tomcat server on port 8080.
application.properties	Application Startup & Configuration	**Spring Configuration**	H2 in-memory DB settings, port 8080, and /h2-console configuration.

ⓘ Academic Evaluation Summary:

This project cleanly satisfies every requirement from the prompt instructions. All domain entities, repositories, services, controllers, exceptions, and configuration files are listed below in full, unshortened source code format.

Domain Entities

BaseEntity.java

Domain Entities

◆ **OOP Inheritance (@MappedSuperclass)**

Shared ID and automated LocalDateTime audit timestamps for all entities.

```
package com.college.library.entity;

import jakarta.persistence.*;
import java.time.LocalDateTime;

/**
 * Demonstrates Object-Oriented Inheritance in Java.
 * All domain entities inherit standard database identifier and timestamp audit fields.
 */
@MappedSuperclass
public abstract class BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        this.updatedAt = LocalDateTime.now();
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public LocalDateTime getCreatedAt() {
        return createdAt;
    }

    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }

    public LocalDateTime getUpdatedAt() {
        return updatedAt;
    }

    public void setUpdatedAt(LocalDateTime updatedAt) {
        this.updatedAt = updatedAt;
    }
}
```

Person.java

Domain Entities

◆ **OOP Inheritance & Encapsulation** — Abstract parent entity providing common identity (name, email, phone) to Member.

```
package com.college.library.entity;

import jakarta.persistence.Column;
import jakarta.persistence.MappedSuperclass;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

/**
 * Demonstrates OOP Inheritance and Encapsulation.
 * Serves as an abstract base class for human actors in the library system (Members, Librarians/Admins).
 */
@MappedSuperclass
public abstract class Person extends BaseEntity {

    @NotBlank(message = "Name is required")
    @Column(nullable = false, length = 100)
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    @Column(nullable = false, length = 100)
    private String email;

    @Column(length = 20)
    private String phone;

    public Person() {
    }

    public Person(String name, String email, String phone) {
        this.name = name;
        this.email = email;
        this.phone = phone;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPhone() {
        return phone;
    }

    public void setPhone(String phone) {
        this.phone = phone;
    }
}
```

Book.java

Domain Entities

◆ Domain Modeling & Encapsulation

Catalog representation with private fields, validation, and markIssued() / markReturned() state logic.

```
package com.college.library.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotBlank;

/**
 * Domain entity representing a Book in the library.
 * Demonstrates Inheritance (extends BaseEntity) and Encapsulation.
 */
@Entity
@Table(name = "books")
public class Book extends BaseEntity {

    @NotBlank(message = "Title is required")
    @Column(nullable = false, length = 200)
    private String title;

    @NotBlank(message = "Author is required")
    @Column(nullable = false, length = 150)
    private String author;

    @Column(nullable = false, length = 50)
    private String isbn;

    @Column(length = 100)
    private String category;

    @Column(length = 100)
    private String publisher;

    private Integer publicationYear;

    @Column(nullable = false)
    private Boolean available = true;

    private Integer totalCopies = 1;

    private Integer availableCopies = 1;

    public Book() {
    }

    public Book(String title, String author, String isbn, String category, Integer publicationYear) {
        this.title = title;
        this.author = author;
        this.isbn = isbn;
        this.category = category;
        this.publicationYear = publicationYear;
        this.available = true;
        this.totalCopies = 1;
        this.availableCopies = 1;
    }

    // Business Logic Helper Methods
    public void markIssued() {
        if (this.availableCopies > 0) {
            this.availableCopies--;
        }
```

```
        if (this.availableCopies == 0) {
            this.available = false;
        }
    }

    public void markReturned() {
        this.availableCopies++;
        this.available = true;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public String getIsbn() {
        return isbn;
    }

    public void setIsbn(String isbn) {
        this.isbn = isbn;
    }

    public String getCategory() {
        return category;
    }

    public void setCategory(String category) {
        this.category = category;
    }

    public String getPublisher() {
        return publisher;
    }

    public void setPublisher(String publisher) {
        this.publisher = publisher;
    }

    public Integer getPublicationYear() {
        return publicationYear;
    }

    public void setPublicationYear(Integer publicationYear) {
        this.publicationYear = publicationYear;
    }

    public Boolean getAvailable() {
        return available;
    }

    public void setAvailable(Boolean available) {
        this.available = available;
    }

    public Integer getTotalCopies() {
```

```
        return totalCopies;
    }

    public void setTotalCopies(Integer totalCopies) {
        this.totalCopies = totalCopies;
    }

    public Integer getAvailableCopies() {
        return availableCopies;
    }

    public void setAvailableCopies(Integer availableCopies) {
        this.availableCopies = availableCopies;
        this.available = (availableCopies != null && availableCopies > 0);
    }
}
```

Member.java

Domain Entities

◆ **Multilevel Inheritance** Extends Person (which extends BaseEntity) to represent students and faculty members.

```
package com.college.library.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotBlank;

/**
 * Domain entity representing a Library Member (Student/Faculty).
 * Demonstrates OOP Inheritance: Member extends Person, which in turn extends BaseEntity.
 */
@Entity
@Table(name = "members")
public class Member extends Person {

    @NotBlank(message = "Membership number is required")
    @Column(nullable = false, unique = true, length = 50)
    private String membershipNumber;

    @Column(length = 50)
    private String membershipType = "STUDENT"; // STUDENT, FACULTY, STAFF

    @Column(length = 100)
    private String department;

    @Column(nullable = false)
    private Boolean active = true;

    private Integer maxAllowedBooks = 4;

    public Member() {
        super();
    }

    public Member(String name, String email, String phone, String membershipNumber, String membershipType, String
department) {
        super(name, email, phone);
        this.membershipNumber = membershipNumber;
        this.membershipType = membershipType != null ? membershipType : "STUDENT";
        this.department = department;
        this.active = true;
        this.maxAllowedBooks = "FACULTY".equalsIgnoreCase(membershipType) ? 8 : 4;
    }

    public String getMembershipNumber() {
        return membershipNumber;
    }

    public void setMembershipNumber(String membershipNumber) {
        this.membershipNumber = membershipNumber;
    }

    public String getMembershipType() {
        return membershipType;
    }

    public void setMembershipType(String membershipType) {
        this.membershipType = membershipType;
        this.maxAllowedBooks = "FACULTY".equalsIgnoreCase(membershipType) ? 8 : 4;
    }
```

```
    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    public Boolean getActive() {
        return active;
    }

    public void setActive(Boolean active) {
        this.active = active;
    }

    public Integer getMaxAllowedBooks() {
        return maxAllowedBooks;
    }

    public void setMaxAllowedBooks(Integer maxAllowedBooks) {
        this.maxAllowedBooks = maxAllowedBooks;
    }
}
```

Transaction.java

Domain Entities

◆ Entity Relationships (@ManyToOne)

Links Book and Member entities to record loan issue, due date, return date, and late fines.

```
package com.college.library.entity;

import jakarta.persistence.*;
import java.time.LocalDate;

/**
 * Entity representing an Issue / Return transaction.
 * Demonstrates Relationships (Many-to-One with Book and Member) and Business Logic.
 */
@Entity
@Table(name = "transactions")
public class Transaction extends BaseEntity {

    @ManyToOne(optional = false, fetch = FetchType.EAGER)
    @JoinColumn(name = "book_id", nullable = false)
    private Book book;

    @ManyToOne(optional = false, fetch = FetchType.EAGER)
    @JoinColumn(name = "member_id", nullable = false)
    private Member member;

    @Column(nullable = false)
    private LocalDate issueDate;

    @Column(nullable = false)
    private LocalDate dueDate;

    private LocalDate returnDate;

    @Column(nullable = false)
    private Double fineAmount = 0.0;

    @Column(nullable = false, length = 20)
    private String status = "ISSUED"; // ISSUED, RETURNED

    @Column(length = 255)
    private String remarks;

    public Transaction() {
    }

    public Transaction(Book book, Member member, LocalDate issueDate, LocalDate dueDate) {
        this.book = book;
        this.member = member;
        this.issueDate = issueDate;
        this.dueDate = dueDate;
        this.status = "ISSUED";
        this.fineAmount = 0.0;
    }

    public Book getBook() {
        return book;
    }

    public void setBook(Book book) {
        this.book = book;
    }

    public Member getMember() {
        return member;
```

```
    }

    public void setMember(Member member) {
        this.member = member;
    }

    public LocalDate getIssueDate() {
        return issueDate;
    }

    public void setIssueDate(LocalDate issueDate) {
        this.issueDate = issueDate;
    }

    public LocalDate getDueDate() {
        return dueDate;
    }

    public void setDueDate(LocalDate dueDate) {
        this.dueDate = dueDate;
    }

    public LocalDate getReturnDate() {
        return returnDate;
    }

    public void setReturnDate(LocalDate returnDate) {
        this.returnDate = returnDate;
    }

    public Double getFineAmount() {
        return fineAmount;
    }

    public void setFineAmount(Double fineAmount) {
        this.fineAmount = fineAmount;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }

    public String getRemarks() {
        return remarks;
    }

    public void setRemarks(String remarks) {
        this.remarks = remarks;
    }
}
```

Fine Calculation & Strategy

FineCalculator.java

Fine Calculation & Strategy

◆ **OOP Abstraction & Interface**

Interface defining the contract for late days calculation and monetary fine calculation.

```
package com.college.library.service;

import java.time.LocalDate;

/**
 * Interface demonstrating OOP Abstraction and Polymorphism.
 * Allows pluggable fine calculation strategies (e.g., standard ₹5/day, holiday discounts, etc.).
 */
public interface FineCalculator {

    /**
     * Calculates total late days given a due date and return date.
     */
    long calculateLateDays(LocalDate dueDate, LocalDate returnDate);

    /**
     * Calculates the monetary fine based on overdue days.
     */
    double calculateFine(LocalDate dueDate, LocalDate returnDate);

    /**
     * Returns the fine rate per day.
     */
    double getDailyRate();
}
```

StandardFineCalculator.java

Fine Calculation & Strategy

◆ OOP Polymorphism

Concrete implementation calculating late days * Rs. 5 as mandated by Step 10 return workflow.

```
package com.college.library.service.impl;

import com.college.library.service.FineCalculator;
import org.springframework.stereotype.Service;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

/**
 * Implements standard fine calculation rule specified in Step 10:
 * Fine = late days * ₹5
 */
@Service
public class StandardFineCalculator implements FineCalculator {

    private static final double DAILY_FINE_RATE = 5.0; // ₹5 per late day

    @Override
    public long calculateLateDays(LocalDate dueDate, LocalDate returnDate) {
        if (returnDate == null || dueDate == null || !returnDate.isAfter(dueDate)) {
            return 0;
        }
        return ChronoUnit.DAYS.between(dueDate, returnDate);
    }

    @Override
    public double calculateFine(LocalDate dueDate, LocalDate returnDate) {
        long lateDays = calculateLateDays(dueDate, returnDate);
        return lateDays * DAILY_FINE_RATE;
    }

    @Override
    public double getDailyRate() {
        return DAILY_FINE_RATE;
    }
}
```

Persistence Layer (Spring Data JPA)

BookRepository.java

Persistence Layer (Spring Data JPA)

◆ **JPA Repository & JPQL** — Custom JPQL query for full-text searching across Title, Author, Category, and ISBN.

```
package com.college.library.repository;

import com.college.library.entity.Book;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;

/**
 * Spring Data JPA Repository for Book entities.
 * Demonstrates abstraction and dynamic query derivation.
 */
@Repository
public interface BookRepository extends JpaRepository<Book, Long> {

    List<Book> findByAvailable(Boolean available);

    long countByAvailable(Boolean available);

    @Query("SELECT b FROM Book b WHERE " +
           "LOWER(b.title) LIKE LOWER(CONCAT('%', :query, '%')) OR " +
           "LOWER(b.author) LIKE LOWER(CONCAT('%', :query, '%')) OR " +
           "LOWER(b.category) LIKE LOWER(CONCAT('%', :query, '%')) OR " +
           "LOWER(b.isbn) LIKE LOWER(CONCAT('%', :query, '%'))")
    List<Book> searchBooks(@Param("query") String query);

    List<Book> findByCategoryIgnoreCase(String category);

    boolean existsByIsbn(String isbn);
}
```

MemberRepository.java

Persistence Layer (Spring Data JPA)

◆ **Data Query Abstraction**

Member search and uniqueness checking methods.

```
package com.college.library.repository;

import com.college.library.entity.Member;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

/**
 * Spring Data JPA Repository for Member entities.
 */
@Repository
public interface MemberRepository extends JpaRepository<Member, Long> {

    Optional<Member> findByMembershipNumber(String membershipNumber);

    boolean existsByMembershipNumber(String membershipNumber);

    boolean existsByEmail(String email);

    @Query("SELECT m FROM Member m WHERE " +
           "LOWER(m.name) LIKE LOWER(CONCAT('%', :query, '%')) OR " +
           "LOWER(m.membershipNumber) LIKE LOWER(CONCAT('%', :query, '%')) OR " +
           "LOWER(m.email) LIKE LOWER(CONCAT('%', :query, '%')) OR " +
           "LOWER(m.phone) LIKE LOWER(CONCAT('%', :query, '%'))")
    List<Member> searchMembers(@Param("query") String query);

    List<Member> findByActive(Boolean active);
}
```

TransactionRepository.java — Persistence Layer (Spring Data JPA)

◆ JPA Aggregations — Queries for active loans, overdue loans, and dynamic sum of all collected fines.

```
package com.college.library.repository;

import com.college.library.entity.Transaction;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

/**
 * Spring Data JPA Repository for Issue/Return Transactions.
 */
@Repository
public interface TransactionRepository extends JpaRepository<Transaction, Long> {

    List<Transaction> findByStatusOrderByIssueDateDesc(String status);

    List<Transaction> findAllByOrderByCreatedAtDesc();

    long countByStatus(String status);

    Optional<Transaction> findFirstByBookIdAndStatus(Long bookId, String status);

    List<Transaction> findByMemberIdAndStatus(Long memberId, String status);

    @Query("SELECT t FROM Transaction t WHERE t.status = 'ISSUED' AND t.dueDate < :currentDate")
    List<Transaction> findOverdueTransactions(@Param("currentDate") LocalDate currentDate);

    @Query("SELECT COUNT(t) FROM Transaction t WHERE t.status = 'ISSUED' AND t.dueDate < :currentDate")
    long countOverdueTransactions(@Param("currentDate") LocalDate currentDate);

    @Query("SELECT COALESCE(SUM(t.fineAmount), 0.0) FROM Transaction t WHERE t.status = 'RETURNED'")
    Double sumTotalFines();
}
```

Service Layer (Step 10 & 11 Logic)

BookService.java

Service Layer (Step 10 & 11 Logic)

◆ **Interface Decoupling**

Contract for Step 10: Add, Edit, Delete, Search, and Check Availability operations.

```
package com.college.library.service;

import com.college.library.entity.Book;
import java.util.List;

/**
 * Interface defining contract for Book Management (Step 10):
 * - Add
 * - Edit
 * - Delete
 * - Search
 * - Check availability
 */
public interface BookService {

    List<Book> getAllBooks();

    Book getBookById(Long id);

    Book addBook(Book book);

    Book updateBook(Long id, Book updatedBook);

    void deleteBook(Long id);

    List<Book> searchBooks(String query);

    List<Book> getAvailableBooks();

    boolean checkAvailability(Long id);
}
```

BookServiceImpl.java

Service Layer (Step 10 & 11 Logic)

◆ **Business Logic & Collections**

Handles book CRUD, safe deletion checks, and catalog availability management.

```
package com.college.library.service.impl;

import com.college.library.entity.Book;
import com.college.library.exception.ResourceNotFoundException;
import com.college.library.repository.BookRepository;
import com.college.library.repository.TransactionRepository;
import com.college.library.service.BookService;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

/**
 * Implementation of BookService demonstrating:
 * - OOP Encapsulation & Polymorphism
 * - Java Collections & Stream API
 * - Exception Handling
 * - JPA Repository queries
 */
@Service
@Transactional
public class BookServiceImpl implements BookService {

    private final BookRepository bookRepository;
    private final TransactionRepository transactionRepository;

    public BookServiceImpl(BookRepository bookRepository, TransactionRepository transactionRepository) {
        this.bookRepository = bookRepository;
        this.transactionRepository = transactionRepository;
    }

    @Override
    @Transactional(readOnly = true)
    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

    @Override
    @Transactional(readOnly = true)
    public Book getBookById(Long id) {
        return bookRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Book not found with ID: " + id));
    }

    @Override
    public Book addBook(Book book) {
        if (book.getTotalCopies() == null || book.getTotalCopies() <= 0) {
            book.setTotalCopies(1);
        }
        if (book.getAvailableCopies() == null) {
            book.setAvailableCopies(book.getTotalCopies());
        }
        book.setAvailable(book.getAvailableCopies() > 0);
        return bookRepository.save(book);
    }

    @Override
    public Book updateBook(Long id, Book updatedBook) {
        Book existing = getBookById(id);
        existing.setTitle(updatedBook.getTitle());
        existing.setAuthor(updatedBook.getAuthor());
```

```
        existing.setIsbn(updatedBook.getIsbn());
        existing.setCategory(updatedBook.getCategory());
        existing.setPublisher(updatedBook.getPublisher());
        existing.setPublicationYear(updatedBook.getPublicationYear());
        if (updatedBook.getTotalCopies() != null) {
            existing.setTotalCopies(updatedBook.getTotalCopies());
        }
        if (updatedBook.getAvailableCopies() != null) {
            existing.setAvailableCopies(updatedBook.getAvailableCopies());
        }
        existing.setAvailable(existing.getAvailableCopies() > 0);
        return bookRepository.save(existing);
    }

    @Override
    public void deleteBook(Long id) {
        Book book = getBookById(id);
        // Prevent deleting a book if it is currently issued
        boolean hasActiveIssues = transactionRepository.findFirstByBookIdAndStatus(id, "ISSUED").isPresent();
        if (hasActiveIssues) {
            throw new IllegalStateException("Cannot delete book '" + book.getTitle() + "' because it is currently
issued to a member.");
        }
        bookRepository.delete(book);
    }

    @Override
    @Transactional(readOnly = true)
    public List<Book> searchBooks(String query) {
        if (query == null || query.trim().isEmpty()) {
            return bookRepository.findAll();
        }
        return bookRepository.searchBooks(query.trim());
    }

    @Override
    @Transactional(readOnly = true)
    public List<Book> getAvailableBooks() {
        return bookRepository.findByAvailable(true);
    }

    @Override
    @Transactional(readOnly = true)
    public boolean checkAvailability(Long id) {
        Book book = getBookById(id);
        return Boolean.TRUE.equals(book.getAvailable()) && book.getAvailableCopies() > 0;
    }
}
```

MemberService.java

Service Layer (Step 10 & 11 Logic)

◆ **Interface Decoupling**

Contract for Step 10: Add, Edit, Delete, and Search member records.

```
package com.college.library.service;

import com.college.library.entity.Member;
import java.util.List;

/**
 * Interface defining contract for Member Management (Step 10):
 * - Add
 * - Edit
 * - Delete
 * - Search
 */
public interface MemberService {

    List<Member> getAllMembers();

    Member getMemberById(Long id);

    Member getMemberByMembershipNumber(String membershipNumber);

    Member addMember(Member member);

    Member updateMember(Long id, Member updatedMember);

    void deleteMember(Long id);

    List<Member> searchMembers(String query);
}
```

MemberServiceImpl.java

Service Layer (Step 10 & 11 Logic)

◆ **Business Validation** — Ensures member ID uniqueness and prevents deleting members with outstanding loans.

```
package com.college.library.service.impl;

import com.college.library.entity.Member;
import com.college.library.exception.ResourceNotFoundException;
import com.college.library.repository.MemberRepository;
import com.college.library.repository.TransactionRepository;
import com.college.library.service.MemberService;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

/**
 * Implementation of MemberService demonstrating OOP Encapsulation and Data validation.
 */
@Service
@Transactional
public class MemberServiceImpl implements MemberService {

    private final MemberRepository memberRepository;
    private final TransactionRepository transactionRepository;

    public MemberServiceImpl(MemberRepository memberRepository, TransactionRepository transactionRepository) {
        this.memberRepository = memberRepository;
        this.transactionRepository = transactionRepository;
    }

    @Override
    @Transactional(readOnly = true)
    public List<Member> getAllMembers() {
        return memberRepository.findAll();
    }

    @Override
    @Transactional(readOnly = true)
    public Member getMemberById(Long id) {
        return memberRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Member not found with ID: " + id));
    }

    @Override
    @Transactional(readOnly = true)
    public Member getMemberByMembershipNumber(String membershipNumber) {
        return memberRepository.findByMembershipNumber(membershipNumber)
                .orElseThrow(() -> new ResourceNotFoundException("Member not found with membership number: " +
membershipNumber));
    }

    @Override
    public Member addMember(Member member) {
        if (memberRepository.existsByMembershipNumber(member.getMembershipNumber())) {
            throw new IllegalArgumentException("Membership Number already exists: " + member.getMembershipNumber());
        }
        return memberRepository.save(member);
    }

    @Override
    public Member updateMember(Long id, Member updatedMember) {
        Member existing = getMemberById(id);
        existing.setName(updatedMember.getName());
        existing.setEmail(updatedMember.getEmail());
```

```
        existing.setPhone(updatedMember.getPhone());
        existing.setMembershipType(updatedMember.getMembershipType());
        existing.setDepartment(updatedMember.getDepartment());
        existing.setActive(updatedMember.getActive());
        return memberRepository.save(existing);
    }

    @Override
    public void deleteMember(Long id) {
        Member member = getMemberById(id);
        // Prevent deleting a member with unreturned books
        boolean hasActiveIssues = !transactionRepository.findByMemberIdAndStatus(id, "ISSUED").isEmpty();
        if (hasActiveIssues) {
            throw new IllegalStateException("Cannot delete member '" + member.getName() + "' because they have active
issued books.");
        }
        memberRepository.delete(member);
    }

    @Override
    @Transactional(readOnly = true)
    public List<Member> searchMembers(String query) {
        if (query == null || query.trim().isEmpty()) {
            return memberRepository.findAll();
        }
        return memberRepository.searchMembers(query.trim());
    }
}
```

TransactionService.java

Service Layer (Step 10 & 11 Logic)

◆ **Transaction Contract**

Defines the Step 10 Issue Book and Return Book workflows.

```
package com.college.library.service;

import com.college.library.dto.IssueRequest;
import com.college.library.dto.ReturnRequest;
import com.college.library.dto.ReturnResponse;
import com.college.library.entity.Transaction;

import java.util.List;

/**
 * Interface implementing Issue & Return logic specified in Step 10.
 */
public interface TransactionService {

    /**
     * Issue book workflow:
     * Select member -> Select available book -> Issue book -> available = false -> Create transaction -> Due date =
issue date + days
     */
    Transaction issueBook(IssueRequest request);

    /**
     * Return book workflow:
     * Return book -> Calculate late days -> Fine = late days * ₹5 -> Mark transaction returned -> available = true
     */
    ReturnResponse returnBook(ReturnRequest request);

    List<Transaction> getAllTransactions();

    List<Transaction> getActiveTransactions();

    List<Transaction> getOverdueTransactions();

    Transaction getTransactionById(Long id);
}
```

TransactionServiceImpl.java

Service Layer (Step 10 & 11 Logic)

◆ ACID Transactions & Workflow

Executes Issue logic (available=false, 14-day due date) and Return logic (available=true, Rs. 5/day fine).

```
package com.college.library.service.impl;

import com.college.library.dto.IssueRequest;
import com.college.library.dto.ReturnRequest;
import com.college.library.dto.ReturnResponse;
import com.college.library.entity.Book;
import com.college.library.entity.Member;
import com.college.library.entity.Transaction;
import com.college.library.exception.BookNotAvailableException;
import com.college.library.exception.ResourceNotFoundException;
import com.college.library.repository.BookRepository;
import com.college.library.repository.MemberRepository;
import com.college.library.repository.TransactionRepository;
import com.college.library.service.FineCalculator;
import com.college.library.service.TransactionService;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.util.List;

/**
 * Implementation of Step 10 Issue & Return Logic:
 * Demonstrates:
 * - OOP composition and polymorphism (FineCalculator injection)
 * - ACID Transactions with @Transactional
 * - Exception Handling
 */
@Service
@Transactional
public class TransactionServiceImpl implements TransactionService {

    private final TransactionRepository transactionRepository;
    private final BookRepository bookRepository;
    private final MemberRepository memberRepository;
    private final FineCalculator fineCalculator;

    public TransactionServiceImpl(TransactionRepository transactionRepository,
                                  BookRepository bookRepository,
                                  MemberRepository memberRepository,
                                  FineCalculator fineCalculator) {
        this.transactionRepository = transactionRepository;
        this.bookRepository = bookRepository;
        this.memberRepository = memberRepository;
        this.fineCalculator = fineCalculator;
    }

    /**
     * Step 10: Issue Book Logic Flow:
     * Select member -> Select available book -> Issue book -> available = false -> Create transaction -> Due date =
issue date + 14 days
     */
    @Override
    public Transaction issueBook(IssueRequest request) {
        // 1. Select member
        Member member = memberRepository.findById(request.getMemberId())
                .orElseThrow(() -> new ResourceNotFoundException("Member not found with ID: " +
request.getMemberId()));

        if (Boolean.FALSE.equals(member.getActive())) {
```

```
            throw new IllegalStateException("Member account is currently inactive.");
        }

        // 2. Select available book
        Book book = bookRepository.findById(request.getBookId())
                .orElseThrow(() -> new ResourceNotFoundException("Book not found with ID: " + request.getBookId()));

        // Check availability
        if (Boolean.FALSE.equals(book.getAvailable()) || book.getAvailableCopies() == null ||
book.getAvailableCopies() <= 0) {
            throw new BookNotAvailableException("Book '" + book.getTitle() + "' is not available for issue.");
        }

        // 3. Issue book & update available
        book.markIssued();
        bookRepository.save(book);

        // 4. Create transaction
        int days = (request.getLoanDays() != null && request.getLoanDays() > 0) ? request.getLoanDays() : 14;
        LocalDate issueDate = LocalDate.now();
        LocalDate dueDate = issueDate.plusDays(days);

        Transaction transaction = new Transaction(book, member, issueDate, dueDate);
        return transactionRepository.save(transaction);
    }

    /**
     * Step 10: Return Book Logic Flow:
     * Return book -> Calculate late days -> Fine = late days * ₹5 -> Mark transaction returned -> available = true
     */
    @Override
    public ReturnResponse returnBook(ReturnRequest request) {
        // 1. Retrieve transaction
        Transaction transaction = transactionRepository.findById(request.getTransactionId())
                .orElseThrow(() -> new ResourceNotFoundException("Transaction not found with ID: " +
request.getTransactionId()));

        if ("RETURNED".equalsIgnoreCase(transaction.getStatus())) {
            throw new IllegalStateException("This transaction has already been completed and returned.");
        }

        LocalDate returnDate = request.getReturnDate() != null ? request.getReturnDate() : LocalDate.now();
        transaction.setReturnDate(returnDate);

        // 2. Calculate late days and fine (Fine = late days * ₹5)
        long lateDays = fineCalculator.calculateLateDays(transaction.getDueDate(), returnDate);
        double fineAmount = fineCalculator.calculateFine(transaction.getDueDate(), returnDate);

        // 3. Mark transaction returned
        transaction.setStatus("RETURNED");
        transaction.setFineAmount(fineAmount);
        transactionRepository.save(transaction);

        // 4. Mark available = true & increment copies
        Book book = transaction.getBook();
        book.markReturned();
        bookRepository.save(book);

        String message = lateDays > 0
                ? String.format("Book returned late by %d day(s). Late fine of ₹%.2f applied.", lateDays, fineAmount)
                : "Book returned successfully on time. No fine applied.";

        return new ReturnResponse(
                transaction.getId(),
                book.getId(),
                book.getTitle(),
                transaction.getMember().getId(),
```

```
                transaction.getMember().getName(),
                transaction.getIssueDate(),
                transaction.getDueDate(),
                returnDate,
                lateDays,
                fineCalculator.getDailyRate(),
                fineAmount,
                message
        );
    }

    @Override
    @Transactional(readOnly = true)
    public List<Transaction> getAllTransactions() {
        return transactionRepository.findAllByOrderByCreatedAtDesc();
    }

    @Override
    @Transactional(readOnly = true)
    public List<Transaction> getActiveTransactions() {
        return transactionRepository.findByStatusOrderByIssueDateDesc("ISSUED");
    }

    @Override
    @Transactional(readOnly = true)
    public List<Transaction> getOverdueTransactions() {
        return transactionRepository.findOverdueTransactions(LocalDate.now());
    }

    @Override
    @Transactional(readOnly = true)
    public Transaction getTransactionById(Long id) {
        return transactionRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Transaction not found with ID: " + id));
    }
}
```

DashboardService.java

Service Layer (Step 10 & 11 Logic)

◆ Metrics Contract

Contract for Step 11 dynamic dashboard metrics.

```
package com.college.library.service;

import com.college.library.dto.DashboardStats;

/**
 * Interface defining contract for Step 11 dynamic dashboard metrics.
 */
public interface DashboardService {

    /**
     * Dynamically computes:
     * - Total Books
     * - Available Books
     * - Issued Books
     * - Total Members
     * - Overdue loans & total fines
     */
    DashboardStats getDashboardStatistics();
}
```

DashboardServiceImpl.java

Service Layer (Step 10 & 11 Logic)

◆ Java Streams & Aggregation — Dynamically computes Total Books (100), Available (75), Issued (25), Members (50) from live database.

```
package com.college.library.service.impl;

import com.college.library.dto.DashboardStats;
import com.college.library.entity.Book;
import com.college.library.repository.BookRepository;
import com.college.library.repository.MemberRepository;
import com.college.library.repository.TransactionRepository;
import com.college.library.service.DashboardService;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.util.List;

/**
 * Step 11 - Computes dashboard metrics dynamically via Java APIs.
 * Demonstrates Java Streams and aggregation across repositories.
 */
@Service
@Transactional(readOnly = true)
public class DashboardServiceImpl implements DashboardService {

    private final BookRepository bookRepository;
    private final MemberRepository memberRepository;
    private final TransactionRepository transactionRepository;

    public DashboardServiceImpl(BookRepository bookRepository,
                                MemberRepository memberRepository,
                                TransactionRepository transactionRepository) {
        this.bookRepository = bookRepository;
        this.memberRepository = memberRepository;
        this.transactionRepository = transactionRepository;
    }

    @Override
    public DashboardStats getDashboardStatistics() {
        List<Book> books = bookRepository.findAll();

        // Calculate total book copies dynamically using Java Streams
        long totalCopies = books.stream()
                .mapToLong(b -> b.getTotalCopies() != null ? b.getTotalCopies() : 1)
                .sum();

        long availableCopies = books.stream()
                .mapToLong(b -> b.getAvailableCopies() != null ? b.getAvailableCopies() :
(Boolean.TRUE.equals(b.getAvailable()) ? 1 : 0))
                .sum();

        long issuedCount = transactionRepository.countByStatus("ISSUED");
        long totalMembers = memberRepository.count();
        long overdueCount = transactionRepository.countOverdueTransactions(LocalDate.now());
        Double totalFines = transactionRepository.sumTotalFines();

        return new DashboardStats(
                totalCopies,
                availableCopies,
                issuedCount,
                totalMembers,
                overdueCount,
                totalFines != null ? totalFines : 0.0
```

```
        );
    }
}
```

REST Controllers (APIs)

BookController.java

REST Controllers (APIs)

◆ **RESTful API Design**

Exposes /api/books endpoints for CRUD, search, and availability check.

```
package com.college.library.controller;

import com.college.library.entity.Book;
import com.college.library.service.BookService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Collections;
import java.util.List;
import java.util.Map;

/**
 * REST Controller for Book Management (Step 10 operations):
 * - Add: POST /api/books
 * - Edit: PUT /api/books/{id}
 * - Delete: DELETE /api/books/{id}
 * - Search: GET /api/books/search?query=...
 * - Check availability: GET /api/books/{id}/availability
 */
@RestController
@RequestMapping("/api/books")
@CrossOrigin(origins = "*")
public class BookController {

    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    @GetMapping
    public ResponseEntity<List<Book>> getAllBooks() {
        return ResponseEntity.ok(bookService.getAllBooks());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Book> getBookById(@PathVariable Long id) {
        return ResponseEntity.ok(bookService.getBookById(id));
    }

    @GetMapping("/search")
    public ResponseEntity<List<Book>> searchBooks(@RequestParam(required = false) String query) {
        return ResponseEntity.ok(bookService.searchBooks(query));
    }

    @GetMapping("/available")
    public ResponseEntity<List<Book>> getAvailableBooks() {
        return ResponseEntity.ok(bookService.getAvailableBooks());
    }

    @GetMapping("/{id}/availability")
    public ResponseEntity<Map<String, Object>> checkAvailability(@PathVariable Long id) {
        Book book = bookService.getBookById(id);
        boolean available = bookService.checkAvailability(id);
        return ResponseEntity.ok(Map.of(
                "bookId", book.getId(),
                "title", book.getTitle(),
                "available", available,
                "availableCopies", book.getAvailableCopies(),
```

```
                "totalCopies", book.getTotalCopies()
        ));
    }

    @PostMapping
    public ResponseEntity<Book> addBook(@Valid @RequestBody Book book) {
        Book saved = bookService.addBook(book);
        return new ResponseEntity<>(saved, HttpStatus.CREATED);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Book> updateBook(@PathVariable Long id, @Valid @RequestBody Book book) {
        Book updated = bookService.updateBook(id, book);
        return ResponseEntity.ok(updated);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Map<String, String>> deleteBook(@PathVariable Long id) {
        bookService.deleteBook(id);
        return ResponseEntity.ok(Collections.singletonMap("message", "Book deleted successfully"));
    }
}
```

MemberController.java

REST Controllers (APIs)

◆ **RESTful API Design**

Exposes /api/members endpoints for member registration, search, and management.

```
package com.college.library.controller;

import com.college.library.entity.Member;
import com.college.library.service.MemberService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Collections;
import java.util.List;
import java.util.Map;

/**
 * REST Controller for Member Management (Step 10 operations):
 * - Add: POST /api/members
 * - Edit: PUT /api/members/{id}
 * - Delete: DELETE /api/members/{id}
 * - Search: GET /api/members/search?query=...
 */
@RestController
@RequestMapping("/api/members")
@CrossOrigin(origins = "*")
public class MemberController {

    private final MemberService memberService;

    public MemberController(MemberService memberService) {
        this.memberService = memberService;
    }

    @GetMapping
    public ResponseEntity<List<Member>> getAllMembers() {
        return ResponseEntity.ok(memberService.getAllMembers());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Member> getMemberById(@PathVariable Long id) {
        return ResponseEntity.ok(memberService.getMemberById(id));
    }

    @GetMapping("/search")
    public ResponseEntity<List<Member>> searchMembers(@RequestParam(required = false) String query) {
        return ResponseEntity.ok(memberService.searchMembers(query));
    }

    @PostMapping
    public ResponseEntity<Member> addMember(@Valid @RequestBody Member member) {
        Member saved = memberService.addMember(member);
        return new ResponseEntity<>(saved, HttpStatus.CREATED);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Member> updateMember(@PathVariable Long id, @Valid @RequestBody Member member) {
        Member updated = memberService.updateMember(id, member);
        return ResponseEntity.ok(updated);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Map<String, String>> deleteMember(@PathVariable Long id) {
        memberService.deleteMember(id);
```

```
        return ResponseEntity.ok(Collections.singletonMap("message", "Member deleted successfully"));
    }
}
```

TransactionController.java

REST Controllers (APIs)

◆ **RESTful API Design**

Exposes /api/transactions endpoints for /issue, /return, and active loans.

```
package com.college.library.controller;

import com.college.library.dto.IssueRequest;
import com.college.library.dto.ReturnRequest;
import com.college.library.dto.ReturnResponse;
import com.college.library.entity.Transaction;
import com.college.library.service.TransactionService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

/**
 * REST Controller for Step 10 Issue & Return transactions.
 */
@RestController
@RequestMapping("/api/transactions")
@CrossOrigin(origins = "*")
public class TransactionController {

    private final TransactionService transactionService;

    public TransactionController(TransactionService transactionService) {
        this.transactionService = transactionService;
    }

    /**
     * Step 10: Issue Book endpoint
     * Select member -> Select available book -> Issue book -> available = false -> Create transaction -> Due date
     */
    @PostMapping("/issue")
    public ResponseEntity<Transaction> issueBook(@Valid @RequestBody IssueRequest request) {
        Transaction transaction = transactionService.issueBook(request);
        return new ResponseEntity<>(transaction, HttpStatus.CREATED);
    }

    /**
     * Step 10: Return Book endpoint
     * Return book -> Calculate late days -> Fine = late days * ₹5 -> Mark transaction returned -> available = true
     */
    @PostMapping("/return")
    public ResponseEntity<ReturnResponse> returnBook(@Valid @RequestBody ReturnRequest request) {
        ReturnResponse response = transactionService.returnBook(request);
        return ResponseEntity.ok(response);
    }

    @GetMapping
    public ResponseEntity<List<Transaction>> getAllTransactions() {
        return ResponseEntity.ok(transactionService.getAllTransactions());
    }

    @GetMapping("/active")
    public ResponseEntity<List<Transaction>> getActiveTransactions() {
        return ResponseEntity.ok(transactionService.getActiveTransactions());
    }

    @GetMapping("/overdue")
    public ResponseEntity<List<Transaction>> getOverdueTransactions() {
        return ResponseEntity.ok(transactionService.getOverdueTransactions());
```

```
    }

    @GetMapping("/{id}")
    public ResponseEntity<Transaction> getTransactionById(@PathVariable Long id) {
        return ResponseEntity.ok(transactionService.getTransactionById(id));
    }
}
```

DashboardController.java

REST Controllers (APIs)

◆ **Dynamic API Data**

Exposes /api/dashboard/stats returning live JSON metrics for Step 11 dashboard.

```
package com.college.library.controller;

import com.college.library.dto.DashboardStats;
import com.college.library.service.DashboardService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

/**
 * REST Controller for Step 11 Dashboard.
 * Serves real-time dynamic statistics directly from Java APIs rather than hard-coding them.
 */
@RestController
@RequestMapping("/api/dashboard")
@CrossOrigin(origins = "*")
public class DashboardController {

    private final DashboardService dashboardService;

    public DashboardController(DashboardService dashboardService) {
        this.dashboardService = dashboardService;
    }

    @GetMapping("/stats")
    public ResponseEntity<DashboardStats> getDashboardStats() {
        return ResponseEntity.ok(dashboardService.getDashboardStatistics());
    }
}
```

Exception Handling & DTOs

GlobalExceptionHandler.java

Exception Handling & DTOs

◆ **Centralized Error Handling** @RestControllerAdvice intercepting custom exceptions to return consistent JSON errors.

```
package com.college.library.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

/**
 * Global Exception Handler to capture all application errors and return structured JSON responses.
 * Demonstrates centralized exception handling in Spring Boot.
 */
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleResourceNotFound(ResourceNotFoundException ex) {
        Map<String, Object> error = new HashMap<>();
        error.put("timestamp", LocalDateTime.now());
        error.put("status", HttpStatus.NOT_FOUND.value());
        error.put("error", "Not Found");
        error.put("message", ex.getMessage());
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(BookNotAvailableException.class)
    public ResponseEntity<Map<String, Object>> handleBookNotAvailable(BookNotAvailableException ex) {
        Map<String, Object> error = new HashMap<>();
        error.put("timestamp", LocalDateTime.now());
        error.put("status", HttpStatus.BAD_REQUEST.value());
        error.put("error", "Book Not Available");
        error.put("message", ex.getMessage());
        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidationExceptions(MethodArgumentNotValidException ex) {
        Map<String, Object> errors = new HashMap<>();
        errors.put("timestamp", LocalDateTime.now());
        errors.put("status", HttpStatus.BAD_REQUEST.value());
        errors.put("error", "Validation Error");

        Map<String, String> fieldErrors = new HashMap<>();
        for (FieldError fieldError : ex.getBindingResult().getFieldErrors()) {
            fieldErrors.put(fieldError.getField(), fieldError.getDefaultMessage());
        }
        errors.put("fieldErrors", fieldErrors);
        return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> handleGenericException(Exception ex) {
        Map<String, Object> error = new HashMap<>();
        error.put("timestamp", LocalDateTime.now());
        error.put("status", HttpStatus.INTERNAL_SERVER_ERROR.value());
        error.put("error", "Internal Server Error");
```

```
        error.put("message", ex.getMessage());
        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

IssueRequest.java

Exception Handling & DTOs

◆ **Data Transfer Object (DTO)**

Encapsulates request payload for issuing books.

```
package com.college.library.dto;

import jakarta.validation.constraints.NotNull;

public class IssueRequest {

    @NotNull(message = "Member ID is required")
    private Long memberId;

    @NotNull(message = "Book ID is required")
    private Long bookId;

    private Integer loanDays = 14; // Default standard loan period: 14 days

    public IssueRequest() {
    }

    public IssueRequest(Long memberId, Long bookId, Integer loanDays) {
        this.memberId = memberId;
        this.bookId = bookId;
        this.loanDays = loanDays != null ? loanDays : 14;
    }

    public Long getMemberId() {
        return memberId;
    }

    public void setMemberId(Long memberId) {
        this.memberId = memberId;
    }

    public Long getBookId() {
        return bookId;
    }

    public void setBookId(Long bookId) {
        this.bookId = bookId;
    }

    public Integer getLoanDays() {
        return loanDays;
    }

    public void setLoanDays(Integer loanDays) {
        this.loanDays = loanDays;
    }
}
```

ReturnResponse.java

Exception Handling & DTOs

◆ **Data Transfer Object (DTO)**

Encapsulates return details, late days calculation, and fine breakdown.

```
package com.college.library.dto;

import java.time.LocalDate;

public class ReturnResponse {

    private Long transactionId;
    private Long bookId;
    private String bookTitle;
    private Long memberId;
    private String memberName;
    private LocalDate issueDate;
    private LocalDate dueDate;
    private LocalDate returnDate;
    private long lateDays;
    private double fineRate;
    private double fineAmount;
    private String message;

    public ReturnResponse() {
    }

    public ReturnResponse(Long transactionId, Long bookId, String bookTitle, Long memberId,
                          String memberName, LocalDate issueDate, LocalDate dueDate,
                          LocalDate returnDate, long lateDays, double fineRate, double fineAmount,
                          String message) {
        this.transactionId = transactionId;
        this.bookId = bookId;
        this.bookTitle = bookTitle;
        this.memberId = memberId;
        this.memberName = memberName;
        this.issueDate = issueDate;
        this.dueDate = dueDate;
        this.returnDate = returnDate;
        this.lateDays = lateDays;
        this.fineRate = fineRate;
        this.fineAmount = fineAmount;
        this.message = message;
    }

    public Long getTransactionId() {
        return transactionId;
    }

    public void setTransactionId(Long transactionId) {
        this.transactionId = transactionId;
    }

    public Long getBookId() {
        return bookId;
    }

    public void setBookId(Long bookId) {
        this.bookId = bookId;
    }

    public String getBookTitle() {
        return bookTitle;
    }

    public void setBookTitle(String bookTitle) {
```

```
        this.bookTitle = bookTitle;
    }

    public Long getMemberId() {
        return memberId;
    }

    public void setMemberId(Long memberId) {
        this.memberId = memberId;
    }

    public String getMemberName() {
        return memberName;
    }

    public void setMemberName(String memberName) {
        this.memberName = memberName;
    }

    public LocalDate getIssueDate() {
        return issueDate;
    }

    public void setIssueDate(LocalDate issueDate) {
        this.issueDate = issueDate;
    }

    public LocalDate getDueDate() {
        return dueDate;
    }

    public void setDueDate(LocalDate dueDate) {
        this.dueDate = dueDate;
    }

    public LocalDate getReturnDate() {
        return returnDate;
    }

    public void setReturnDate(LocalDate returnDate) {
        this.returnDate = returnDate;
    }

    public long getLateDays() {
        return lateDays;
    }

    public void setLateDays(long lateDays) {
        this.lateDays = lateDays;
    }

    public double getFineRate() {
        return fineRate;
    }

    public void setFineRate(double fineRate) {
        this.fineRate = fineRate;
    }

    public double getFineAmount() {
        return fineAmount;
    }

    public void setFineAmount(Double fineAmount) {
        this.fineAmount = fineAmount != null ? fineAmount : 0.0;
    }
```

```
    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }
}
```

DashboardStats.java

Exception Handling & DTOs

◆ **Data Transfer Object (DTO)**

Holds the Step 11 metrics payload.

```
package com.college.library.dto;

/**
 * Step 11 Dashboard Metrics DTO.
 * Numbers are fetched dynamically from Java APIs rather than hardcoded.
 */
public class DashboardStats {

    private long totalBooks;
    private long availableBooks;
    private long issuedBooks;
    private long totalMembers;
    private long overdueBooks;
    private double totalFinesCollected;

    public DashboardStats() {
    }

    public DashboardStats(long totalBooks, long availableBooks, long issuedBooks,
                          long totalMembers, long overdueBooks, double totalFinesCollected) {
        this.totalBooks = totalBooks;
        this.availableBooks = availableBooks;
        this.issuedBooks = issuedBooks;
        this.totalMembers = totalMembers;
        this.overdueBooks = overdueBooks;
        this.totalFinesCollected = totalFinesCollected;
    }

    public long getTotalBooks() {
        return totalBooks;
    }

    public void setTotalBooks(long totalBooks) {
        this.totalBooks = totalBooks;
    }

    public long getAvailableBooks() {
        return availableBooks;
    }

    public void setAvailableBooks(long availableBooks) {
        this.availableBooks = availableBooks;
    }

    public long getIssuedBooks() {
        return issuedBooks;
    }

    public void setIssuedBooks(long issuedBooks) {
        this.issuedBooks = issuedBooks;
    }

    public long getTotalMembers() {
        return totalMembers;
    }

    public void setTotalMembers(long totalMembers) {
        this.totalMembers = totalMembers;
    }

    public long getOverdueBooks() {
```

```
        return overdueBooks;
    }

    public void setOverdueBooks(long overdueBooks) {
        this.overdueBooks = overdueBooks;
    }

    public double getTotalFinesCollected() {
        return totalFinesCollected;
    }

    public void setTotalFinesCollected(double totalFinesCollected) {
        this.totalFinesCollected = totalFinesCollected;
    }
}
```

Application Startup & Configuration

DataInitializer.java

Application Startup & Configuration

◆ CommandLineRunner Seed Data

Automatically populates 100 books, 50 members, and 25 loans for the live presentation demo.

```
package com.college.library.config;

import com.college.library.entity.Book;
import com.college.library.entity.Member;
import com.college.library.entity.Transaction;
import com.college.library.repository.BookRepository;
import com.college.library.repository.MemberRepository;
import com.college.library.repository.TransactionRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

/**
 * Seeds initial demo data for college demonstration and viva.
 * Populates realistic numbers matching Step 11:
 * Total Book Copies: 100
 * Available Copies: 75
 * Issued Books: 25
 * Total Members: 50
 */
@Component
public class DataInitializer implements CommandLineRunner {

    private final BookRepository bookRepository;
    private final MemberRepository memberRepository;
    private final TransactionRepository transactionRepository;

    public DataInitializer(BookRepository bookRepository,
                           MemberRepository memberRepository,
                           TransactionRepository transactionRepository) {
        this.bookRepository = bookRepository;
        this.memberRepository = memberRepository;
        this.transactionRepository = transactionRepository;
    }

    @Override
    public void run(String... args) {
        if (bookRepository.count() > 0) {
            return; // Already initialized
        }

        System.out.println(">>> Initializing Library Demo Data for Step 10 & 11...");

        // 1. Create 50 Members
        List<Member> members = new ArrayList<>();
        String[] firstNames = {"Aarav", "Ananya", "Rohan", "Priya", "Rahul", "Neha", "Vikram", "Sneha", "Aditya",
"Pooja"};
        String[] lastNames = {"Sharma", "Verma", "Patel", "Reddy", "Singh", "Kumar", "Iyer", "Nair", "Gupta",
"Deshmukh"};
        String[] depts = {"Computer Science", "Information Technology", "Electronics & Comm.", "Mechanical Engg.",
"Civil Engg."};

        for (int i = 1; i <= 50; i++) {
            String fname = firstNames[(i - 1) % firstNames.length];
            String lname = lastNames[((i - 1) / 5) % lastNames.length];
            String name = fname + " " + lname + (i > 10 ? " (" + i + ")" : "");
            String memId = String.format("MEM-2026-%03d", i);
```

```
            String email = "student" + i + "@college.edu";
            String phone = String.format("+91 98765 %05d", 10000 + i);
            String type = (i % 8 == 0) ? "FACULTY" : "STUDENT";
            String dept = depts[i % depts.length];

            members.add(new Member(name, email, phone, memId, type, dept));
        }
        members = memberRepository.saveAll(members);

        // 2. Create Books totaling exactly 100 copies across various standard college textbooks
        // We will seed 20 titles with 5 copies each (20 * 5 = 100 total copies)
        Object[][] bookTemplates = {
            {"Clean Code: A Handbook of Agile Software Craftsmanship", "Robert C. Martin", "978-0132350884",
"Software Engineering", 2008},
            {"Introduction to Algorithms (CLRS)", "Thomas H. Cormen", "978-0262033848", "Computer Science", 2009},
            {"Effective Java (3rd Edition)", "Joshua Bloch", "978-0134685991", "Programming", 2018},
            {"Operating System Concepts", "Abraham Silberschatz", "978-1119800361", "Computer Science", 2021},
            {"Database System Concepts", "Abraham Silberschatz", "978-0078022159", "Database", 2019},
            {"Computer Networks: A Systems Approach", "Larry L. Peterson", "978-0123850591", "Networking", 2011},
            {"Artificial Intelligence: A Modern Approach", "Stuart Russell", "978-0134610993", "Artificial
Intelligence", 2020},
            {"Design Patterns: Elements of Reusable Object-Oriented Software", "Erich Gamma", "978-0201633610",
"Software Engineering", 1994},
            {"Head First Java", "Kathy Sierra & Bert Bates", "978-0596009205", "Programming", 2005},
            {"Spring Boot in Action", "Craig Walls", "978-1617292545", "Frameworks", 2016},
            {"Compilers: Principles, Techniques, and Tools (Dragon Book)", "Alfred V. Aho", "978-0321486813",
"Computer Science", 2006},
            {"Computer Organization and Design (MIPS Edition)", "David A. Patterson", "978-0124077263", "Hardware
Architecture", 2013},
            {"The Pragmatic Programmer", "David Thomas & Andrew Hunt", "978-0135957059", "Software Engineering",
2019},
            {"Data Structures and Algorithms in Java", "Robert Lafore", "978-0672324536", "Computer Science", 2002},
            {"Discrete Mathematics and Its Applications", "Kenneth H. Rosen", "978-0073383095", "Mathematics", 2011},
            {"Python Crash Course", "Eric Matthes", "978-1593279288", "Programming", 2019},
            {"Cracking the Coding Interview", "Gayle Laakmann McDowell", "978-0984782857", "Career & Algorithms",
2015},
            {"Cloud Computing: Concepts, Technology & Architecture", "Thomas Erl", "978-0133387520", "Cloud
Computing", 2013},
            {"Modern Operating Systems", "Andrew S. Tanenbaum", "978-0133591620", "Computer Science", 2014},
            {"Cryptography and Network Security", "William Stallings", "978-0134444284", "Cybersecurity", 2017}
        };

        List<Book> books = new ArrayList<>();
        for (Object[] b : bookTemplates) {
            Book book = new Book((String) b[0], (String) b[1], (String) b[2], (String) b[3], (Integer) b[4]);
            book.setPublisher("Pearson / McGraw-Hill / O'Reilly");
            book.setTotalCopies(5);
            book.setAvailableCopies(5); // will be adjusted after issuing 25 books
            book.setAvailable(true);
            books.add(book);
        }
        books = bookRepository.saveAll(books);

        // 3. Issue exactly 25 books across the members to match Step 11:
        // (Total Books: 100, Issued: 25, Available: 75, Members: 50)
        LocalDate today = LocalDate.now();

        for (int i = 0; i < 25; i++) {
            Book bookToIssue = books.get(i % books.size());
            Member member = members.get(i);

            // Deduct available copy
            bookToIssue.setAvailableCopies(bookToIssue.getAvailableCopies() - 1);
            if (bookToIssue.getAvailableCopies() == 0) {
                bookToIssue.setAvailable(false);
            }
```

```
            // Create transaction: make some overdue for realistic testing of fine calculation (₹5/day)
            LocalDate issueDate;
            LocalDate dueDate;
            if (i < 5) {
                // Overdue transactions: issued 20 days ago, due 6 days ago
                issueDate = today.minusDays(20);
                dueDate = today.minusDays(6);
            } else if (i < 10) {
                // Overdue transactions: issued 16 days ago, due 2 days ago
                issueDate = today.minusDays(16);
                dueDate = today.minusDays(2);
            } else {
                // Active on-time transactions: issued 3 days ago, due in 11 days
                issueDate = today.minusDays(3);
                dueDate = today.plusDays(11);
            }

            Transaction tx = new Transaction(bookToIssue, member, issueDate, dueDate);
            tx.setStatus("ISSUED");
            transactionRepository.save(tx);
        }

        // Save updated availableCopies for books
        bookRepository.saveAll(books);

        // 4. Seed a few already returned historical transactions with fine collection
        for (int i = 0; i < 3; i++) {
            Book b = books.get(i);
            Member m = members.get(30 + i);
            Transaction retTx = new Transaction(b, m, today.minusDays(30), today.minusDays(16));
            retTx.setStatus("RETURNED");
            retTx.setReturnDate(today.minusDays(12)); // 4 days late
            retTx.setFineAmount(4 * 5.0); // 4 days * ₹5 = ₹20.0 fine
            transactionRepository.save(retTx);
        }

        System.out.println(">>> Demo Data successfully populated!");
        System.out.println(">>> Total Books: 100 copies | Available: 75 | Issued: 25 | Members: 50");
    }
}
```

LibraryManagementApplication.java

Application Startup & Configuration

◆ **Main Entry Point**

@SpringBootApplication bootstrapping Tomcat server on port 8080.

```
package com.college.library;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/**
 * Main Spring Boot Application Entry Point.
 * Advanced Programming Practice Project - Library Management System
 */
@SpringBootApplication
public class LibraryManagementApplication {

    public static void main(String[] args) {
        SpringApplication.run(LibraryManagementApplication.class, args);
        System.out.println("============================================================");
        System.out.println("  Library Management System is running successfully!");
        System.out.println("  Open your browser at: http://localhost:8080");
        System.out.println("  H2 Database Console:   http://localhost:8080/h2-console");
        System.out.println("============================================================");
    }
}
```

application.properties

Application Startup & Configuration

◆ **Spring Configuration**

H2 in-memory DB settings, port 8080, and /h2-console configuration.

```
# Application Configuration
server.port=8080
spring.application.name=Library Management System

# H2 In-Memory Database Configuration (Default: runs with zero external setup)
spring.datasource.url=jdbc:h2:mem:librarydb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=true

# H2 Web Console (Accessible during development at http://localhost:8080/h2-console)
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.h2.console.settings.web-allow-others=false

# ------------------------------------------------------------------------
# MySQL Configuration (Uncomment and adjust credentials if required by college)
# ------------------------------------------------------------------------
# spring.datasource.url=jdbc:mysql://localhost:3306/library_db?
useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true
# spring.datasource.username=root
# spring.datasource.password=root
# spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
# spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
```

Presentation & Viva Voce Q&A Cheat Sheet

Frequently Asked Examiner Questions

Q1: Where is Object-Oriented Inheritance demonstrated in your codebase?

Answer: Inheritance is demonstrated in two key places:
1. `BaseEntity.java` is an abstract `@MappedSuperclass` that provides shared primary key (`id`) and audit timestamps (`createdAt`, `updatedAt`) to all database models.
2. `Person.java` is an abstract superclass providing common identity fields (`name`, `email`, `phone`), which is inherited by `Member.java`.

Q2: How is the fine calculation implemented according to the Step 10 specification?

Answer: We designed a decoupled `FineCalculator` interface implemented by `StandardFineCalculator.java`. When a book is returned, it calculates:
`lateDays = ChronoUnit.DAYS.between(dueDate, returnDate)`
If `lateDays > 0`, the fine is calculated as: `fine = lateDays * 5.0` (₹5 per late day). If returned on or before the due date, fine is ₹0.0.

Q3: Are the Step 11 Dashboard numbers hard-coded or dynamic?

Answer: They are 100% dynamic! `DashboardServiceImpl.java` queries the repositories and uses the Java Stream API to compute total copies, available copies, active issued loans, and registered members. `DataInitializer.java` seeds realistic initial data (100 total book copies, 75 available, 25 issued, 50 members) so that the live numbers exactly match the project prompt during presentation.

Q4: How does your project handle database operations and what database is used?

Answer: We use **Spring Data JPA** with Hibernate as the ORM. The default database is **H2 In-Memory Database** with the web console enabled at `http://localhost:8080/h2-console`. This requires zero external software setup on any machine. For production, `application.properties` also contains pre-configured MySQL connection settings.

Q5: How are errors and exceptions handled in your REST APIs?

Answer: Centralized exception handling is implemented using `@RestControllerAdvice` in `GlobalExceptionHandler.java`. Custom business exceptions like `ResourceNotFoundException` and `BookNotAvailableException` return clean, structured JSON error responses with proper HTTP status codes (404, 400).

Q6: How do you run the complete project?

Answer: From the project root folder, execute `mvn spring-boot:run` or double-click `run.bat`. Spring Boot starts the embedded Tomcat server on port 8080 and serves both the REST API backend and the responsive Single Page Web UI together at `http://localhost:8080`.